

Cours Développement Web

Niveau intermédiaire • Atelier – Activité n°2

Interactivité : JavaScript avancé, DOM et événements

Durée : 05 heures | Modalité : individuel | Remise : dossier compressé

Objectifs	Renforcer l'interactivité en JavaScript via le DOM et les événements ; consolider les bonnes pratiques (validation, feedback utilisateur, persistance LocalStorage).
Ressources	Ressources du module + fichiers de ressources fournis (HTML/CSS) + recherche Internet si nécessaire.
Type de l'activité	Activité individuelle.
Outils de communication	Forum (questions, partage de ressources, assistance tuteur).
Résultat attendu	Un fichier .zip contenant tous les fichiers HTML/CSS/JS réalisés, ainsi qu'un court document explicatif (ou README).
Durée / Évaluation	Durée estimée : 06 heures. Évaluation par le tuteur (qualité de mise en œuvre + compréhension).

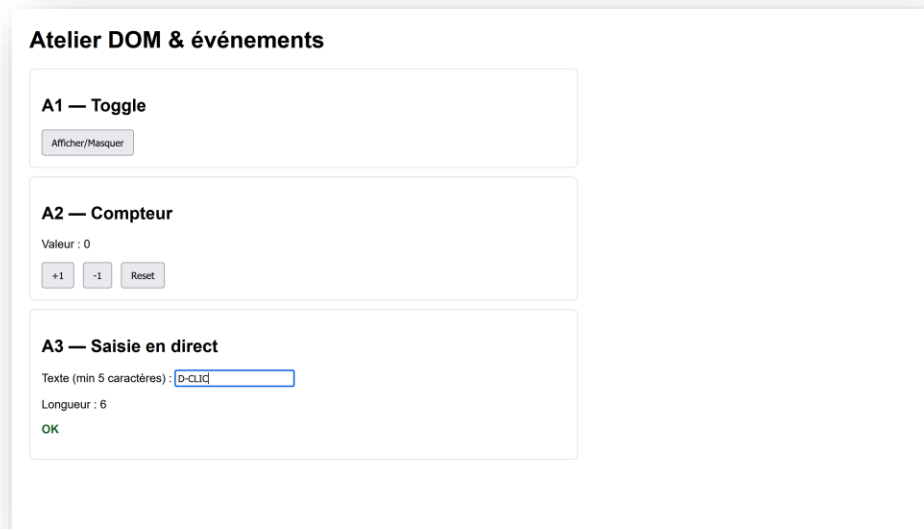
1. Présentation

Cette semaine porte sur l'interactivité : manipulation du DOM, gestion des événements et validation côté client. L'activité est structurée en deux volets :

- Volet A (nouveau) : mini-atelier guidé sur les événements (boutons, affichage/masquage, compteur, formulaire simple) avec solutions au fur et à mesure.
- Volet B (activité officielle) : activité fournie dans l'espace de cours, conservée intégralement.

2. Volet A — Mini-atelier guidé : DOM + événements (avec solutions)

Objectif : comprendre et pratiquer les événements (click, input, change, submit) et la manipulation du DOM (textContent, classList, style). Vous pouvez copier/coller les solutions pour tester, puis modifier pour vous entraîner. Vous obtenez à la fin de cette activité le résultat suivant :



2.1 Mise en place (fichiers)

- Créez un dossier : 00_Atelier_Evenements/
- Créez 3 fichiers : index.html, style.css, app.js.
- Liez style.css dans <head> et app.js en bas du <body>.

Solution — squelette minimal

```
<!-- 00_Atelier_Evenements/index.html -->
<!doctype html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Atelier DOM & événements</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Atelier DOM & événements</h1>
  <main class="container">
    <!-- Les exercices seront ajoutés ici -->
  </main>

  <script src="app.js"></script>
</body>
</html>

/* 00_Atelier_Evenements/style.css */
body{font-family: Arial, sans-serif; margin: 24px;}
.container{max-width: 760px;}
.bloc{border: 1px solid #ddd; padding: 16px; border-radius: 8px; margin: 12px 0;}
```

```
.btn{padding: 8px 12px; cursor: pointer; margin-right: 8px;}
.hidden{display:none;}
.msg{font-weight: 700; margin-top: 10px;}
.error{color:#b00020;}
.ok{color:#1b5e20;}
```

```
// 00_Atelier_Evenements/app.js
console.log('Atelier chargé ');
```

2.2 Exercice A1 — Bouton : afficher / masquer un contenu

- Ajouter un paragraphe caché + 1 bouton 'Afficher/Masquer'.

- Au clic : basculer l'affichage du paragraphe (class hidden).

Vous pouvez utiliser la méthode toggle.

toggle est une méthode de classList qui agit comme un **interrupteur** pour une classe donnée:

- Si la classe **n'est pas présente** sur l'élément, toggle l'**ajoute**.
- Si la classe **est déjà présente**, toggle la **supprime**.

classList est un objet spécial qui contient toutes les classes CSS appliquées à l'élément

Solution A1 (HTML)

```
<!-- À ajouter dans <main> -->
<section class="bloc" id="exA1">
  <h2>A1 – Toggle</h2>
  <button class="btn" id="btnToggle">Afficher/Masquer</button>
  <p id="texteToggle" class="hidden">Bonjour ! Ce contenu peut être affiché ou masqué.</p>
</section>
```

Solution A1 (JS)

```
// A1 – Toggle
const btnToggle = document.getElementById('btnToggle');
const texteToggle = document.getElementById('texteToggle');

btnToggle.addEventListener('click', function(){
  texteToggle.classList.toggle('hidden');
});
```

2.3 Exercice A2 — Compteur : + / - / reset

- Afficher un nombre (0).
- 3 boutons : +1, -1, Reset.
- Mettre à jour l'affichage au clic.

Solution A2 (HTML)

```
<section class="bloc" id="exA2">
  <h2>A2 – Compteur</h2>
  <p>Valeur : <span id="count">0</span></p>
  <button class="btn" id="plus">+1</button>
  <button class="btn" id="moins">-1</button>
  <button class="btn" id="reset">Reset</button>
</section>
```

Solution A2 (JS)

```
// A2 – Compteur
let valeur = 0;
const spanCount = document.getElementById('count');

document.getElementById('plus').addEventListener('click', function(){
  valeur++;
  spanCount.textContent = valeur;
});

document.getElementById('moins').addEventListener('click', function(){
  valeur--;
  spanCount.textContent = valeur;
});

document.getElementById('reset').addEventListener('click', function(){
  valeur = 0;
  spanCount.textContent = valeur;
});
```

2.4 Exercice A3 — Champ texte : message en direct (événement input)

- Ajouter un champ <input>.
- Afficher en direct le nombre de caractères.
- Afficher un message d'erreur si < 5 caractères.

Solution A3 (HTML)

```
<section class="bloc" id="exA3">
  <h2>A3 – Saisie en direct</h2>
  <label for="txt">Texte (min 5 caractères) :</label>
  <input id="txt" type="text" placeholder="Tapez ici...">
  <p>Longueur : <span id="len">0</span></p>
  <p id="msgA3" class="msg"></p>
</section>
```

Solution A3 (JS)

```
// A3 – input
const txt = document.getElementById('txt');
const len = document.getElementById('len');
```

```

const msgA3 = document.getElementById('msgA3');

txt.addEventListener('input', function(){
  const n = txt.value.length;
  len.textContent = n;

  if(n < 5){
    msgA3.textContent = 'Erreur : minimum 5 caractères.';
    msgA3.className = 'msg error';
  } else {
    msgA3.textContent = 'OK';
    msgA3.className = 'msg ok';
  }
});

```

2.5 Exercice A4 — Mini-formulaire : submit + preventDefault

- Créer un mini-formulaire (nom + email).
- Au submit : empêcher le rechargement (preventDefault).
- Valider : nom $\geq$ 3 caractères, email contient '@'.
- Afficher un message final (OK ou erreur).

Solution A4 (HTML)

```

<section class="bloc" id="exA4">
  <h2>A4 – Formulaire</h2>
  <form id="formA4">
    <div>
      <label for="nom">Nom :</label>
      <input id="nom" type="text" required>
    </div>
    <div>
      <label for="email">Email :</label>
      <input id="email" type="text" required>
    </div>
    <button class="btn" type="submit">Envoyer</button>
  </form>
  <p id="msgA4" class="msg"></p>
</section>

```

Solution A4 (JS)

```

// A4 – submit
const formA4 = document.getElementById('formA4');
const msgA4 = document.getElementById('msgA4');

formA4.addEventListener('submit', function(e){
  e.preventDefault();

  const nom = document.getElementById('nom').value.trim();
  const email = document.getElementById('email').value.trim();

```

```

if(nom.length < 3){
    msgA4.textContent = 'Erreur : nom trop court.';
    msgA4.className = 'msg error';
    return;
}

if(!email.includes('@')){
    msgA4.textContent = 'Erreur : email invalide.';
    msgA4.className = 'msg error';
    return;
}

msgA4.textContent = 'Données valides ✔';
msgA4.className = 'msg ok';
});

```

2.6 À retenir (résumé)

- Un événement = une action (click, input, submit...).
- `addEventListener` associe une action à une fonction.
- `classList.toggle()` permet d'afficher/masquer facilement.
- `preventDefault()` évite le rechargement lors de l'envoi d'un formulaire.

3. Volet B — Activité principale

Consignes :

1. **Téléchargement des Ressources** : Téléchargez les fichiers de ressources fournis, comprenant un fichier HTML et un fichier CSS. Assurez-vous de les conserver dans un seul dossier.

Client

Civilité

☐ Mademoiselle
☒ Madame
☐ Monsieur

Nom / Prénom

Adresse

No postal / Localité

Pays

Tunisie
Ajouter pays
Libye
Ajouter pays

Plateforme(s)

☒ Windows
☒ Macintosh
☐ Unix

Applications(s)

Bureautique
DAO
Statistiques
SGBD
Internet

Soumettre formulaire
Vérification formulaire
Effacer

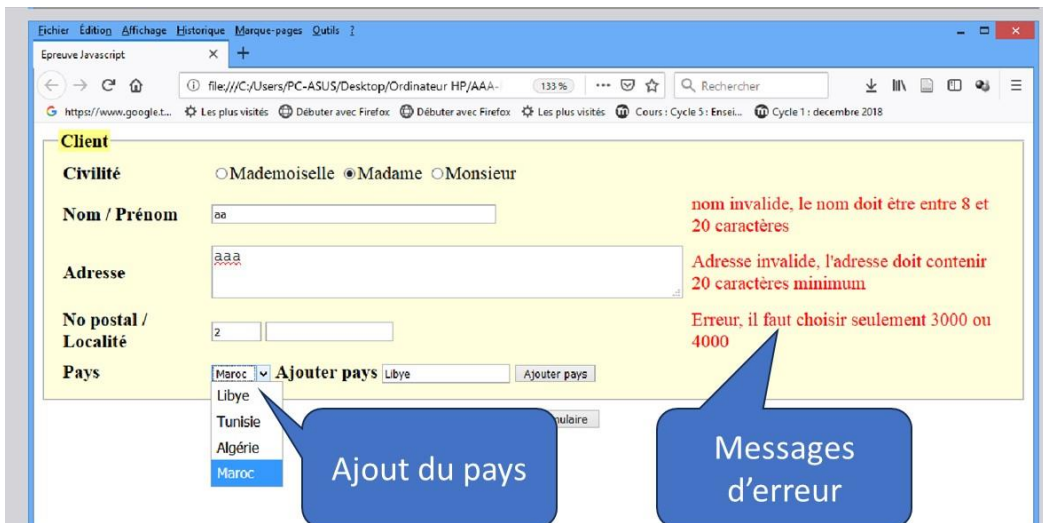
2. **Intégration du CSS** : Intégrez le fichier CSS dans le fichier HTML en utilisant la balise appropriée. Assurez-vous que les styles définis dans le fichier CSS sont appliqués correctement aux éléments HTML.

3. **Fonction AjouterPays()** : Complétez le code de la fonction JavaScript AjouterPays() dans le fichier HTML. Cette fonction doit permettre d'ajouter le nom de la ville se trouvant dans le champ "ajouter pays" à la liste des pays se trouvant dans la liste modifiable "pays". Un élément déjà existant ne doit pas être ajouté en double.

4. **Fonction VerifierFormulaire()** : Complétez le code de la fonction JavaScript VerifierFormulaire() dans le fichier HTML. Cette fonction doit vérifier les éléments saisis dans le formulaire et afficher des messages d'erreurs dans des zones appropriées pour montrer le type d'erreur :

- Le champ "nom et prénoms" est invalide si la taille n'est pas entre 8 et 20 caractères.
- Le champ "adresse" est invalide si la taille est inférieure à 20 caractères.
- Le "No postal" est invalide si la valeur est différente de 3000 et de 4000.
- Si la valeur est 3000, affichez dans le champ "localité" "ville1", et si la valeur est 4000, affichez "ville 2".

En plus de l'affichage du message d'erreur, le nom du champ doit changer de couleur en rouge. Si l'utilisateur corrige l'erreur et donne une valeur valide, le message d'erreur doit disparaître, et la couleur du champ revient en noir.



5. **Fonction AfficherAlert()** : Complétez le code de la fonction JavaScript AfficherAlert(). Cette fonction doit afficher une boîte de dialogue (alert) avec les valeurs saisies dans le formulaire si les données sont valides. Si des erreurs sont détectées, affichez un message d'erreur dans la boîte de dialogue pour inviter l'utilisateur à corriger les erreurs.

Client

Civilité: ☐ Mademoiselle ☐ Madame ☒ Monsieur

Nom / Prénom: Léna Moussa

Adresse: Avenue des princes
CP 3000
Niger

No postal / Localité: 3000 Ville 1

Afficher les informations saisies dans le formulaire

Modal dialog content:
Léna Moussa
Avenue des princes
CP 3000
Niger
Ville 1
OK

6. **Utilisation de l'API Local Storage - Enregistrement** : Ajoutez un bouton "Enregistrer" à votre formulaire. Lorsque l'utilisateur clique sur ce bouton, utilisez l'API Local Storage pour enregistrer les données saisies si elles sont correctes. Les données enregistrées doivent être stockées de manière appropriée pour une récupération ultérieure.

7. **Utilisation de l'API Local Storage - Récupération** : Ajoutez un bouton "Récupérer" à votre formulaire. Lorsque l'utilisateur clique sur ce bouton, utilisez l'API Local Storage pour récupérer les données déjà enregistrées. Affichez ces données directement dans le formulaire s'il y en a, sinon affichez un message d'erreur pour informer l'utilisateur qu'il n'y a pas de données enregistrées.

Une fois que vous avez terminé ces étapes, rassemblez tous les fichiers HTML, CSS et JavaScript dans un dossier, puis compressez le dossier dans un fichier ZIP.

4. Livrables & dépôt

- Un seul fichier .zip : NOM_Prenom_S2_Intermediaire_DOM_Evenements.zip
- Le .zip contient : 00_Atelier_Evenements/ (Volet A) + dossier de l'activité officielle (Volet B) + éventuellement un README.

5. Critères d'évaluation (barème indicatif /20)

Critère	Attendu	Points
Volet A — Événements	Exercices A1–A4 fonctionnels + compréhension démontrée.	0–5
AjouterPays()	Ajout sans doublons, manipulation DOM correcte.	0–4
Validation formulaire	Règles respectées + messages + couleurs dynamiques.	0–6
Alert + LocalStorage	Enregistrer/Récupérer + feedback correct.	0–4
Organisation	Projet propre, zip correct, fichiers bien nommés.	0–1